

Analysis

Jim Denne

2026-05-17

Setup

```
library(knitr)

set.seed(1)
source("src/data_calibration.R")
source("src/severity_models.R")
source("src/reinsurance.R")
source("src/confidence_intervals.R")
source("src/ruin_simulation.R")

# Load and calibrate data

raw <- load_fremtpl2(
  freq_path = "data/freMTPL2freq.rda",
  sev_path  = "data/freMTPL2sev.rda"
)

freq <- extract_claim_frequency(raw$freq)
sev <- extract_claim_amounts(raw$sev)

emp_model <- empirical_severity_model(sev)

# Since lambda is annual, t is measured in years.
t_years <- 5

# Chosen business assumptions
portfolio_size <- 1000      # keeps simulation computationally manageable
capital_multiple <- 0.5     # c0 = 0.5x annual expected claims
theta <- 0.10              # insurer premium loading
xi <- 0.15                 # reinsurer loading, assumed > insurer loading
ruin_limit <- 0.01         # 1% 5-year ruin threshold

lambda_portfolio <- portfolio_size * freq$lambda_per_policy
annual_expected_claims <- lambda_portfolio * emp_model$mean
c0 <- capital_multiple * annual_expected_claims

# Calibration summary -----
calibration_summary <- data.frame(
  Quantity = c(
    "Total claims",
```

```

    "Total exposure",
    "Lambda per policy",
    "Portfolio size",
    "Lambda portfolio",
    "Severity mean",
    "Annual expected claims",
    "Initial capital",
    "Reinsurance loading theta",
    "Safety loading xi"
  ),
  Value = c(
    freq$total_claims,
    freq$total_exposure,
    freq$lambda_per_policy,
    portfolio_size,
    lambda_portfolio,
    emp_model$mean,
    annual_expected_claims,
    c0,
    theta,
    xi
  )
)

kable(
  calibration_summary,
  digits = 4,
  caption = "Calibration inputs and fitted portfolio quantities"
)

```

Table 1: Calibration inputs and fitted portfolio quantities

Quantity	Value
Total claims	26444.0000
Total exposure	358482.8355
Lambda per policy	0.0738
Portfolio size	1000.0000
Lambda portfolio	73.7664
Severity mean	2265.5126
Annual expected claims	167118.7867
Initial capital	83559.3933
Reinsurance loading theta	0.1000
Safety loading xi	0.1500

```

# Severity summary -----
severity_summary <- data.frame(
  Metric = c(
    "Number of claims",
    "Mean severity",
    "Median severity",
    "90th percentile",
    "95th percentile",

```

```

    "99th percentile",
    "Maximum severity"
  ),
  Value = c(
    length(sev),
    mean(sev),
    median(sev),
    as.numeric(quantile(sev, 0.90)),
    as.numeric(quantile(sev, 0.95)),
    as.numeric(quantile(sev, 0.99)),
    max(sev)
  )
)

kable(
  severity_summary,
  digits = 2,
  caption = "Empirical severity distribution summary"
)

```

Table 2: Empirical severity distribution summary

Metric	Value
Number of claims	26444.00
Mean severity	2265.51
Median severity	1172.00
90th percentile	2767.60
95th percentile	4765.09
99th percentile	16451.22
Maximum severity	4075400.56

Ruin Simulation

```

set.seed(1)
# Baseline no-reinsurance run
n_baseline <- 50000

baseline <- evaluate_policy(
  n_sim = n_baseline,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  policy = "none",
  c0 = c0,
  theta = theta,
  xi = 0
)

# Display results
baseline_table <- baseline

```

```

baseline_table$alpha <- ifelse(is.na(baseline_table$alpha), "-", baseline_table$alpha)
baseline_table$d <- ifelse(is.na(baseline_table$d), "-", baseline_table$d)

kable(
  baseline_table,
  digits = 4,
  col.names = c(
    "Policy",
    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Baseline policy performance"
)

```

Table 3: Baseline policy performance

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower	none	-	-	0.2739	0.27	0.2778	83559.39

Reinsurance Optimisation

```

select_best_contract_ci <- function(results, ruin_limit = 0.01) {
  feasible <- results[results$ci_upper <= ruin_limit, ]

  if (nrow(feasible) == 0) {
    return(NULL)
  }

  feasible[which.max(feasible$expected_profit), ]
}

print_candidates <- function(...) {
  candidates <- Filter(Negate(is.null), list(...))
  if (length(candidates) == 0) return(NULL)
  do.call(rbind, candidates)
}

format_candidates <- function(x) {
  if (is.null(x) || nrow(x) == 0) {
    return(data.frame(Message = "No feasible contracts found"))
  }

  x$alpha <- ifelse(is.na(x$alpha), "-", x$alpha)
  x$d <- ifelse(is.na(x$d), "-", x$d)

  x
}

```

Rough Grid Search

```
set.seed(1)

n_grid_rough <- 10000

# Net profit condition for proportional
#  $\alpha > 1 - \theta / \xi$ 
alpha_min_npc <- max(0, 1 - theta / xi)

alpha_grid_rough <- seq(
  from = alpha_min_npc + 0.01,
  to = 1.00,
  by = 0.025
)

# Rough EoL grid:
# Mix empirical quantiles with evenly spaced values.
d_grid_rough <- sort(unique(c(
  as.numeric(quantile(sev, probs = seq(0.50, 0.995, length.out = 35), type = 8)),
  seq(1000, 20000, by = 1000)
))))

# Rough proportional grid search -----
prop_results_rough <- grid_search_proportional(
  alpha_grid = alpha_grid_rough,
  n_sim = n_grid_rough,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  c0 = c0,
  theta = theta,
  xi = xi
)

# Rough EoL grid search -----
eol_results_rough <- grid_search_eol(
  d_grid = d_grid_rough,
  n_sim = n_grid_rough,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  c0 = c0,
  theta = theta,
  xi = xi
)

best_prop_point_rough <- select_best_contract(prop_results_rough,
                                             ruin_limit = ruin_limit)
best_eol_point_rough <- select_best_contract(eol_results_rough,
                                             ruin_limit = ruin_limit)

best_prop_ci_rough <- select_best_contract_ci(prop_results_rough,
                                              ruin_limit = ruin_limit)
```

```

best_eol_ci_rough <- select_best_contract_ci(eol_results_rough,
                                             ruin_limit = ruin_limit)

# Display results -----
rough_point_candidates <- format_candidates(print_candidates(
  best_prop_point_rough,
  best_eol_point_rough
))

rough_ci_candidates <- format_candidates(print_candidates(
  best_prop_ci_rough,
  best_eol_ci_rough
))

kable(
  rough_point_candidates,
  digits = 4,
  col.names = c(
    "Policy",
    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Rough best contracts under point estimate constraint"
)

```

Table 4: Rough best contracts under point estimate constraint

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower25	EoL	-	9000	0.0087	0.0071	0.0107	39125.22

```

kable(
  rough_ci_candidates,
  digits = 4,
  col.names = c(
    "Policy",
    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Rough best contracts under Wilson upper bound constraint"
)

```

Table 5: Rough best contracts under Wilson upper bound constraint

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower24	EoL	-	8000	0.0058	0.0045	0.0075	37820.04

Fine Grid Search

```

set.seed(1)

n_grid_fine <- 50000

# Proportional fine grid -----
# If rough search found a feasible proportional contract, refine near it.
if (!is.null(best_prop_point_rough)) {
  alpha_centre <- best_prop_point_rough$alpha

  alpha_grid_fine <- seq(
    from = max(alpha_min_npc + 0.001, alpha_centre - 0.075),
    to = min(1.00, alpha_centre + 0.075),
    by = 0.005
  )

  prop_results_fine <- grid_search_proportional(
    alpha_grid = alpha_grid_fine,
    n_sim = n_grid_fine,
    t = t_years,
    lambda = lambda_portfolio,
    sev_model = emp_model,
    c0 = c0,
    theta = theta,
    xi = xi
  )

} else {
  cat("\nNo feasible proportional contract found in rough search.\n")
  prop_results_fine <- prop_results_rough
}

##
## No feasible proportional contract found in rough search.
# EoL fine grid -----
# Prefer conservative rough candidate if available; otherwise use point estimate.
eol_centre <- if (!is.null(best_eol_ci_rough)) {
  best_eol_ci_rough$d
} else if (!is.null(best_eol_point_rough)) {
  best_eol_point_rough$d
} else {
  median(d_grid_rough)
}

d_grid_fine <- seq(
  from = max(100, eol_centre - 3000),

```

```

    to = eol_centre + 3000,
    by = 250
  )

eol_results_fine <- grid_search_eol(
  d_grid = d_grid_fine,
  n_sim = n_grid_fine,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  c0 = c0,
  theta = theta,
  xi = xi
)

grid_results <- rbind(prop_results_fine, eol_results_fine)

best_prop_point <- select_best_contract(
  prop_results_fine,
  ruin_limit = ruin_limit
)

best_eol_point <- select_best_contract(
  eol_results_fine,
  ruin_limit = ruin_limit
)

best_prop_conservative <- select_best_contract_ci(
  prop_results_fine,
  ruin_limit = ruin_limit
)

best_eol_conservative <- select_best_contract_ci(
  eol_results_fine,
  ruin_limit = ruin_limit
)

# Display results -----
fine_point_candidates <- format_candidates(print_candidates(
  best_prop_point,
  best_eol_point
))

fine_ci_candidates <- format_candidates(print_candidates(
  best_prop_conservative,
  best_eol_conservative
))

kable(
  fine_point_candidates,
  digits = 4,
  col.names = c(
    "Policy",

```



```

    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Fine best contracts under point estimate constraint"
)

```

Table 6: Fine best contracts under point estimate constraint

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower18	EoL	-	9500	0.0092	0.0084	0.0101	39710.63

```

kable(
  fine_ci_candidates,
  digits = 4,
  col.names = c(
    "Policy",
    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Fine best contracts under Wilson upper bound constraint"
)

```

Table 7: Fine best contracts under Wilson upper bound constraint

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower16	EoL	-	9000	0.0081	0.0074	0.009	39125.22

Final Values

```

prop_candidate <- if (!is.null(best_prop_conservative)) {
  best_prop_conservative
} else {
  best_prop_point
}

eol_candidate <- if (!is.null(best_eol_conservative)) {
  best_eol_conservative
} else {
  best_eol_point
}

evaluate_candidate <- function(candidate, n_final = 200000) {

```

```

if (is.null(candidate)) return(NULL)

evaluate_policy(
  n_sim = n_final,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  policy = candidate$policy,
  c0 = c0,
  theta = theta,
  xi = xi,
  d = if (is.na(candidate$d)) NULL else candidate$d,
  alpha = if (is.na(candidate$alpha)) NULL else candidate$alpha
)
}

# Final Results -----
set.seed(1)
n_final <- 200000

baseline_final <- evaluate_policy(
  n_sim = n_final,
  t = t_years,
  lambda = lambda_portfolio,
  sev_model = emp_model,
  policy = "none",
  c0 = c0,
  theta = theta,
  xi = 0
)

prop_final <- evaluate_candidate(prop_candidate, n_final = n_final)
eol_final <- evaluate_candidate(eol_candidate, n_final = n_final)

final_results <- do.call(rbind, Filter(Negate(is.null), list(
  baseline_final,
  prop_final,
  eol_final
)))

format_final_results <- function(x) {
  x$alpha <- ifelse(is.na(x$alpha), "-", x$alpha)
  x$d <- ifelse(is.na(x$d), "-", x$d)
  x
}

final_comparison_table <- format_final_results(final_results)

kable(
  final_comparison_table,
  digits = 4,
  col.names = c(
    "Policy",

```

```

    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit"
  ),
  caption = "Final reinsurance comparison"
)

```

Table 8: Final reinsurance comparison

	Policy	Alpha	Deductible	Ruin Probability	CI Lower	CI Upper	Expected Profit
lower	none	-	-	0.2735	0.2715	0.2754	83559.39
lower1	EoL	-	9000	0.0085	0.0081	0.0089	39125.22

```

# Capital efficiency summary -----
baseline_profit <- baseline_final$expected_profit
baseline_ruin   <- baseline_final$ruin_prob

final_results$profit_sacrifice <- baseline_profit - final_results$expected_profit
final_results$ruin_reduction   <- baseline_ruin - final_results$ruin_prob

final_results$capital_efficiency <- ifelse(
  final_results$profit_sacrifice > 0,
  final_results$ruin_reduction / final_results$profit_sacrifice,
  NA
)

final_efficiency_table <- format_final_results(final_results)

kable(
  final_efficiency_table,
  digits = 8,
  col.names = c(
    "Policy",
    "Alpha",
    "Deductible",
    "Ruin Probability",
    "CI Lower",
    "CI Upper",
    "Expected Profit",
    "Profit Sacrifice",
    "Ruin Reduction",
    "Capital Efficiency"
  ),
  caption = "Final results with capital efficiency"
)

```

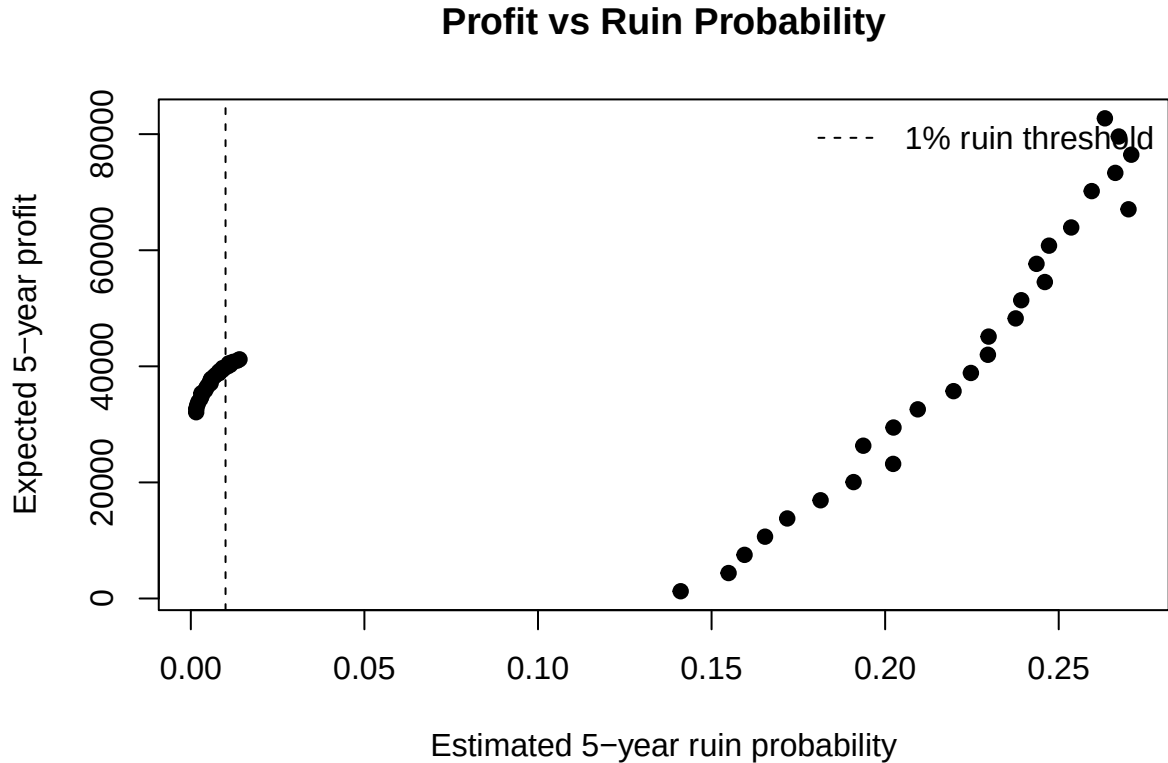
Table 9: Final results with capital efficiency

	Policy	Alpha	Deductible	Ruin Probabil- ity	CI Lower	CI Upper	Expected Profit	Profit Sacrifice	Ruin Re- duction	Capital Efficiency
lower	none	-	-	0.27345	0.271500	0.275407	83559.39	0.00	0.00000	NA
lower1	EoL	-	9000	0.00850	0.008107	0.008911	89125.22	44434.18	0.26495	5.96e-06

Plots

```
# Profit vs ruin probability -----
plot(
  grid_results$ruin_prob,
  grid_results$expected_profit,
  pch = 19,
  xlab = "Estimated 5-year ruin probability",
  ylab = "Expected 5-year profit",
  main = "Profit vs Ruin Probability"
)
abline(v = ruin_limit, lty = 2)

legend(
  "topright",
  legend = c("1% ruin threshold"),
  lty = 2,
  bty = "n"
)
```

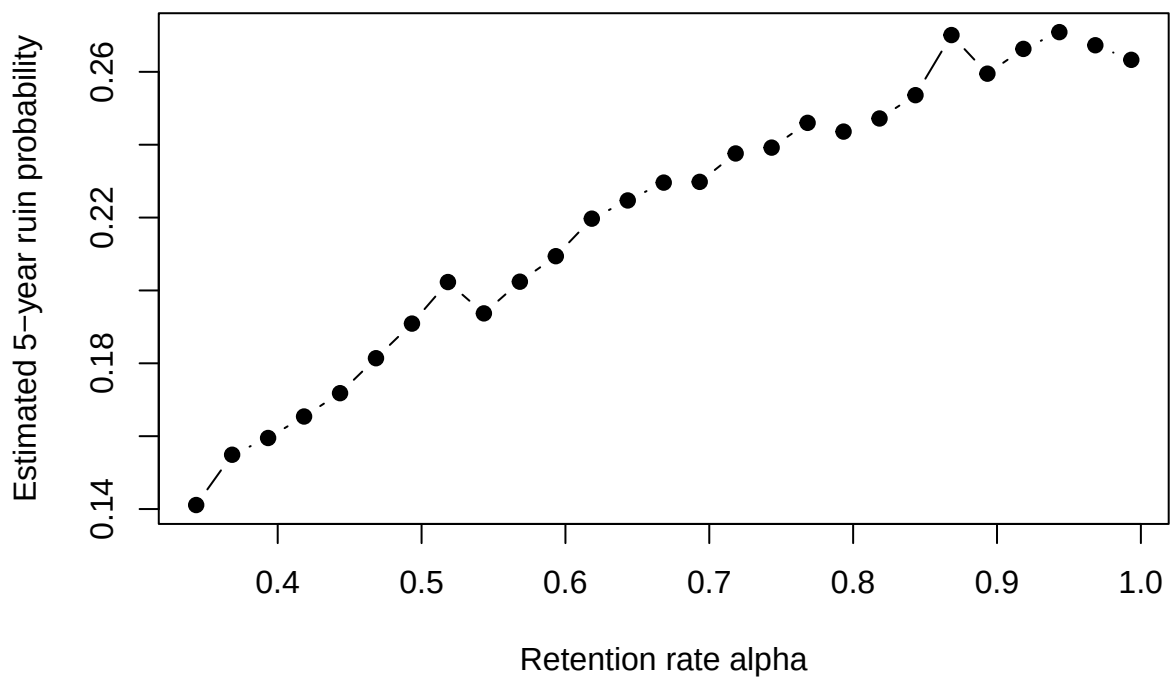


```

# Proportional: alpha vs ruin -----
plot(
  prop_results_fine$alpha,
  prop_results_fine$ruin_prob,
  type = "b",
  pch = 19,
  xlab = "Retention rate alpha",
  ylab = "Estimated 5-year ruin probability",
  main = "Proportional Reinsurance: Ruin vs Retention"
)
abline(h = ruin_limit, lty = 2)

```

Proportional Reinsurance: Ruin vs Retention



```

# EoL: deductible vs ruin -----
plot(
  eol_results_fine$d,
  eol_results_fine$ruin_prob,
  type = "b",
  pch = 19,
  xlab = "Deductible d",
  ylab = "Estimated 5-year ruin probability",
  main = "EoL Reinsurance: Ruin vs Deductible"
)
abline(h = ruin_limit, lty = 2)

```

EoL Reinsurance: Ruin vs Deductible

